Quack Quack What? — Build & Deployment Document

Version: 1.0
Date: March 2026
Author: Taashi Manyanga

---

1. Technology Stack

1.1 Frontend

Technology	Version	Purpose
React	19	UI framework
Tailwind CSS	4	Utility-first styling
Wouter	latest	Client-side routing
TanStack Query	latest	Server state management
Framer Motion	latest	Animations and transitions
Radix UI	latest	Accessible UI primitives
Lucide React	latest	Icon library

1.2 Backend

Technology	Version	Purpose
Express	5	HTTP server framework
Node.js	20+	Runtime environment
TypeScript	latest	Type-safe development
Drizzle ORM	latest	Type-safe database queries
Passport.js	latest	Authentication middleware
openid-client	latest	OIDC protocol implementation
connect-pg-simple	latest	PostgreSQL session store
Zod	latest	Runtime schema validation

1.3 Database

Technology	Purpose
PostgreSQL	Primary data store
Drizzle Kit	Schema migrations (push mode)

1.4 Build Tools

Technology	Purpose
Vite	7
esbuild	Backend bundler (production)
tsx	TypeScript execution (development)

---

2. Project Structure

```
quack-quack-what/
+-- client/
|   +-- index.html                    # HTML entry point
|   +-- src/
|       +-- App.tsx                    # Root component + routing
|       +-- index.css                  # Tailwind v4 config + theme
|       +-- main.tsx                   # React entry point
|       +-- pages/
|       |   +-- landing.tsx            # Guest landing page
|       |   +-- home.tsx               # Main evaluator dashboard
|       |   +-- history.tsx            # User evaluation history
|       |   +-- readme.tsx             # Documentation page
|       |   +-- not-found.tsx          # 404 page
|       +-- components/
|       |   +-- evaluator/
|       |   |   +-- SkillImporter.tsx  # Multi-source import UI
|       |   |   +-- AnalysisSim.tsx    # Analysis animation
|       |   |   +-- EvaluationReport.tsx # Results display
|       |   |   +-- SkillChat.tsx      # LLM chat interface
|       |   |   +-- SettingsModal.tsx  # AI provider config
|       |   +-- ui/                    # Radix-based UI primitives
|       +-- hooks/
|       |   +-- use-auth.ts            # Authentication hook
|       +-- lib/
|           +-- queryClient.ts         # TanStack Query config
|           +-- utils.ts               # Utility functions
|           +-- auth-utils.ts          # Auth error helpers
+-- server/
|   +-- index.ts                       # Express bootstrap
|   +-- db.ts                          # Database connection pool
|   +-- routes.ts                      # API route definitions
|   +-- storage.ts                     # CRUD operations
|   +-- analyzer.ts                    # Static analysis engine
|   +-- llm.ts                         # LLM provider adapter
|   +-- static.ts                      # Production static serving
|   +-- vite.ts                        # Dev Vite middleware
|   +-- replit_integrations/
|       +-- auth/
|           +-- index.ts               # Module exports
|           +-- replitAuth.ts          # OIDC + Passport setup
|           +-- storage.ts             # User CRUD
|           +-- routes.ts              # Auth API routes
+-- shared/
|   +-- schema.ts                      # Drizzle schema + types
|   +-- models/
|       +-- auth.ts                    # Auth table definitions
+-- docs/                              # Documentation suite
+-- drizzle.config.ts                  # Drizzle Kit configuration
+-- tsconfig.json                      # TypeScript configuration
+-- vite.config.ts                     # Vite configuration
+-- tailwind.config.ts                 # Tailwind configuration
+-- package.json                       # Dependencies
```

```
+-- replit.md                          # Project documentation
```

---

3. Environment Variables

Variable	Required	Description
`DATABASE_URL`	Yes	PostgreSQL connection string
`SESSION_SECRET`	Yes	Express session encryption key
`REPL_ID`	Yes	Replit app identifier (auto-set)
`ISSUER_URL`	No	OIDC issuer (defaults to `https://replit.com/oidc`)
`PORT`	No	Server port (defaults to 5000)
`NODE_ENV`	No	Environment mode (development/production)

---

4. Development Setup

4.1 Start Development Server

```
npm run dev
```

This runs `tsx server/index.ts` which starts: - Express API server on port 5000 - Vite dev server as middleware (for HMR)

4.2 Database Schema Push

```
npm run db:push
```

Pushes the current Drizzle schema to the PostgreSQL database without generating migration files.

4.3 Key npm Scripts

Script	Command	Purpose
`dev`	`NODE_ENV=development tsx server/index.ts`	Start dev server
`build`	`vite build && esbuild server/index.ts --bundle --platform=node --outdir=dist`	Production build
`db:push`	`drizzle-kit push`	Push schema to database

---

5. Build Process

5.1 Frontend Build (Vite)

1. TypeScript compilation via esbuild (within Vite)
2. Tailwind CSS processing
3. Asset optimization and code splitting
4. Output to `dist/public/`

5.2 Backend Build (esbuild)

1. TypeScript compilation
2. Bundle all server code into single file
3. Externalize node_modules
4. Output to `dist/`

5.3 Production Serving

In production (`NODE_ENV=production`): - Express serves the built frontend from `dist/public/` - API routes handle backend requests - No Vite dev server or HMR

---

6. Database Management

6.1 Schema Definition

All tables are defined in `shared/schema.ts` (app tables) and `shared/models/auth.ts` (auth tables).

6.2 Schema Changes

1. Modify the schema file(s)
2. Run `npm run db:push`
3. Drizzle Kit compares current DB state and applies changes

6.3 Auth Tables (Protected)

The `users` and `sessions` tables are mandatory for Replit Auth and must not be dropped.

---

7. API Endpoints Reference

7.1 Authentication

Method	Path	Auth	Description
GET	`/api/login`	No	Initiates OIDC login
GET	`/api/callback`	No	OIDC callback handler
GET	`/api/logout`	No	Destroys session, redirects
GET	`/api/auth/user`	Yes	Returns current user profile

7.2 Evaluation

Method	Path	Auth	Description
POST	`/api/fetch-skill`	No	Fetches code from GitHub/URL
POST	`/api/evaluate`	No	Runs analysis, stores result
GET	`/api/evaluations`	Yes	User's evaluations list
GET	`/api/evaluations/:id`	Partial	Single evaluation (ownership check)
GET	`/api/history`	Yes	User's evaluation history

7.3 Configuration

Method	Path	Auth	Description
GET	`/api/provider-config`	No	Get AI provider config (keys masked)
POST	`/api/provider-config`	No	Save/update AI provider config

7.4 Chat

Method	Path	Auth	Description
POST	`/api/chat`	No	Send message to configured LLM

---

8. Deployment

8.1 Platform

- **Host:** Replit
- **Domain:** `*.replit.app` or custom domain
- **SSL:** Automatic via Replit

8.2 Deployment Steps

1. Build: `npm run build`
2. Replit handles deployment, health checks, and TLS
3. Database is persistent across deployments

8.3 Health Check

Express serves on port 5000 (Replit's expected port). The platform monitors the process and restarts on failure.

---

9. Testing Strategy

9.1 Data Test IDs

All interactive and data-display elements have `data-testid` attributes following the pattern: - Interactive: `{action}-{target}` (e.g., `button-submit`, `input-email`) - Display: `{type}-{content}` (e.g., `text-username`, `status-payment`) - Dynamic: `{type}-{description}-{id}` (e.g., `card-evaluation-${id}`)

9.2 Manual Testing Checklist

- ☐ Guest can access home page and run evaluation
- ☐ Guest sees "Guest mode" banner
- ☐ Sign in redirects to Replit auth
- ☐ Authenticated user sees profile in header
- ☐ Evaluations are linked to authenticated user
- ☐ History page shows user's past evaluations
- ☐ LLM chat works with configured provider
- ☐ Report download generates correct file
- ☐ Documentation page renders all 12 sections

---

10. Limitations & Known Issues

1. **GitHub API Rate Limits:** Unauthenticated GitHub API requests are limited to 60/hour
2. **Static Analysis Only:** No runtime execution or dynamic analysis
3. **Single Provider Config:** Only one LLM provider can be active at a time
4. **No Evaluation Deletion:** Users cannot delete past evaluations
5. **Provider Config is Global:** Not per-user (shared across all users)
